Documentação do Projeto NI1 – Space Invaders (POO)

Oct 6, 2026 · @André

Identificação do trabalho

Este documento registra o desenvolvimento do jogo Space Invaders para a NI1 de Programação Orientada a Objetos, incluindo o uso de IA em cada etapa.

O enunciado pede os itens abaixo. A coluna da direita mostra onde cada um é atendido.

Requisito do enunciado	Como foi atendido
Jogo, de preferência em JavaScript com canvas	Space Invaders em JavaScript puro com `<canvas>`
Classes e objetos	`Entidade`, `Jogador`, `Inimigo`, `Projetil`, `Jogo` e os três tipos de inimigo
Herança	`Jogador`, `Inimigo` e `Projetil` herdam de `Entidade`; os três tipos de inimigo herdam de `Inimigo`
Ao menos um polimorfismo	`mover()` e `tentarAtirar()` sobrescritos nas subclasses de `Inimigo`
Código organizado, comentado e indentado	Um arquivo por classe, com comentários explicando cada decisão
Ao menos duas IAs, com comparação	Claude e Gemini (seção de comparação)
Progresso documentado	Este documento

Ideação: prompt 1 e resultado

A primeira conversa com o Claude serviu para escolher o jogo. Foi enviado o enunciado da NI1 (print em PDF) e a pergunta abaixo.

Prompt 1 (Claude):

> *Quais ideias vc acha bom para a criação desse projeto? O professor falou que pode ser bem simples*

O Claude leu o enunciado e sugeriu cinco jogos, todos com várias entidades parecidas e comportamentos diferentes, o que deixa herança e polimorfismo naturais no código.

Ideia	Estrutura de classes proposta
Space Invaders / shooter simples	`Inimigo` base com `InimigoBasico`, `InimigoRapido` e `InimigoAtirador`; `Jogador`, `Projetil`, `Jogo`
Plataforma curta (fuja dos obstáculos)	`Obstaculo` base com `ObstaculoFixo`, `ObstaculoMovel` e `ObstaculoQueCai`; `Personagem`
Arena RPG por turnos	`Personagem` base com `Guerreiro`, `Mago` e `Arqueiro`, cada um com `atacar()` próprio
Pong / Breakout com variações	`Bloco` base com `BlocoNormal`, `BlocoResistente` e `BlocoExplosivo`
Tiro top-down por ondas	`Entidade` base com `Jogador` e `Inimigo`; inimigos com comportamentos diferentes

A IA recomendou o RPG por turnos ou o Space Invaders, porque os dois mostram herança e polimorfismo com pouco trabalho de física e colisão, sobrando tempo para documentar.

Decisão: Space Invaders no VS Code

O grupo escolheu o Space Invaders e perguntou se o VS Code seria suficiente.

Prompt 2 (Claude):

> *Vamos com o space invaders, mas só para confirmar, o vs code daria conta normal?*

Resultado: sim. O projeto é só HTML e JavaScript com `<canvas>`, então não exige engine, Node.js nem servidor backend. A IA recomendou a extensão Live Server para abrir o `index.html` com recarregamento automático. Ela também avisou que módulos ES6 (`import`/`export`) só funcionam com servidor local, e não abrindo o arquivo direto (`file://`). Por isso o projeto usa tags `<script>` comuns, em ordem.

Estrutura proposta:

```
space-invaders/
├── index.html
├── style.css
└── js/
    ├── Entidade.js
    ├── Projetil.js
    ├── Jogador.js
    ├── Inimigo.js
    ├── InimigoBasico.js
    ├── InimigoRapido.js
    ├── InimigoAtirador.js
    └── Jogo.js
```

Geração do código

Resultado: o Claude criou 10 arquivos (HTML, CSS e 8 classes em JavaScript) e conferiu a sintaxe de cada `.js` com `node --check`, sem erros. Essa checagem só valida a sintaxe; o jogo ainda não tinha sido aberto no navegador.

Arquivo	Herda de	Responsabilidade
`Entidade.js`	nenhuma	Classe base: posição, tamanho, cor, `desenhar()`, `atualizar()` e `colideCom()`
`Projetil.js`	`Entidade`	Tiro do jogador (sobe) ou do inimigo (desce); some ao sair da tela
`Jogador.js`	`Entidade`	Move-se com as setas, atira com a barra de espaço (intervalo de 15 frames) e tem 3 vidas
`Inimigo.js`	`Entidade`	Base dos invasores: pontos, direção, `mover()` e `tentarAtirar()` (vazio por padrão)
`InimigoBasico.js`	`Inimigo`	Vale 10 pontos; só anda em bloco, herda tudo
`InimigoRapido.js`	`Inimigo`	Vale 20 pontos; sobrescreve `mover()` para andar com o dobro da velocidade
`InimigoAtirador.js`	`Inimigo`	Vale 30 pontos; sobrescreve `tentarAtirar()` com chance de 0,2% por frame
`Jogo.js`	nenhuma	Loop principal, teclado, onda de 4 linhas com 8 inimigos,

Arquivo	Herda de	Responsabilidade
		colisões, pontos, vidas e fim de jogo
`index.html` / `style.css`	não se aplica	Página com o `<canvas>` de 800 x 600, HUD e carregamento dos scripts em ordem

Contribuição do André: tela de início e tela de fim de jogo

O jogo começava sozinho ao abrir a página e, ao ganhar ou perder, mostrava apenas uma mensagem de texto, e o jogador precisava recarregar a página para jogar de novo. A contribuição do André foi criar uma tela de início e uma tela de fim (vitória ou derrota), com botões para iniciar e para jogar novamente. O código foi gerado com a ajuda do Claude e testado no navegador.

Campo	Descrição
Tipo	Melhoria, que também corrige uma falha de usabilidade
Sintoma	O jogo iniciava sozinho ao abrir a página. Ao perder, aparecia "Recarregue a página para jogar de novo", e não havia botão de reiniciar
Causa	O `Jogo` criava os objetos e chamava `requestAnimationFrame` direto no construtor. A função `_finalizar()` só escrevia uma mensagem e parava o loop, sem como recomeçar
Solução	Três momentos no jogo: tela de início, jogando e tela de fim. Uma classe nova, `Tela`, mostra e esconde as mensagens. O `Jogo` ganhou os métodos `iniciar()` e `_resetarEstado()`
Arquivos alterados	`Jogo.js`, `index.html` e `style.css`
Arquivo novo	`Tela.js`
IA usada	Claude

O que já existia e o que é novo. O sistema de pontos (10, 20 e 30 por tipo de inimigo), o contador de vidas e as condições de vitória e derrota já estavam no código gerado antes. A contribuição foi a tela de início, a tela de fim com o botão de jogar novamente, a legenda de pontos na tela de início, a exibição da pontuação final e o reinício que zera o estado da partida.

Código antes (`Jogo.js`): o jogo iniciava no construtor e o fim só mostrava uma mensagem:

```
  constructor() {
    // ...
```

```
    this.teclas = {};
    this._configurarTeclado();
    this._criarOnda();
    this._loop = this._loop.bind(this);
    requestAnimationFrame(this._loop); // inicia sozinho
  }

  _verificarFimDeJogo() {
    if (this.jogador.vidas <= 0) {
      this._finalizar('Você perdeu! Recarregue a página para jogar de novo.');
    }
    // ...
  }

  _finalizar(texto) {
    this.rodando = false;
    document.getElementById('mensagem').textContent = texto;
  }
```

Código depois (`Jogo.js`): o jogo só começa em `iniciar()`, e o fim chama a tela com o botão:

```
  constructor() {
    // ...
    this.rodando = false; // parado até o jogador clicar
    this.tela = new Tela(() => this.iniciar());
    this._resetarEstado();
    this._desenhar();
    this.tela.mostrarInicio(); // não inicia o loop
  }

  iniciar() {
    if (this.rodando) return; // evita criar dois loops
    this._resetarEstado();    // zera pontos, vidas, inimigos e tiros
    this.tela.esconder();
    this.rodando = true;
    this.idFrame = requestAnimationFrame(this._loop);
  }

  _finalizar(venceu, motivo) {
    this.rodando = false;
    cancelAnimationFrame(this.idFrame);
    this.tela.mostrarFim(venceu, motivo, this.pontos);
  }
```

Classe nova (`Tela.js`), que só cuida do que aparece por cima do jogo:

```
class Tela {
  constructor(aoClicar) {
    this.overlay = document.getElementById('overlay');
    // ... demais elementos
    this.botao.addEventListener('click', aoClicar);
  }
  mostrarInicio() { /* título, instruções, legenda de pontos e botão */ }
  mostrarFim(venceu, motivo, pontos) { /* vitória ou derrota, pontuação final e botão
*/ }
  esconder() { /* some com a tela quando a partida começa */ }
}
```

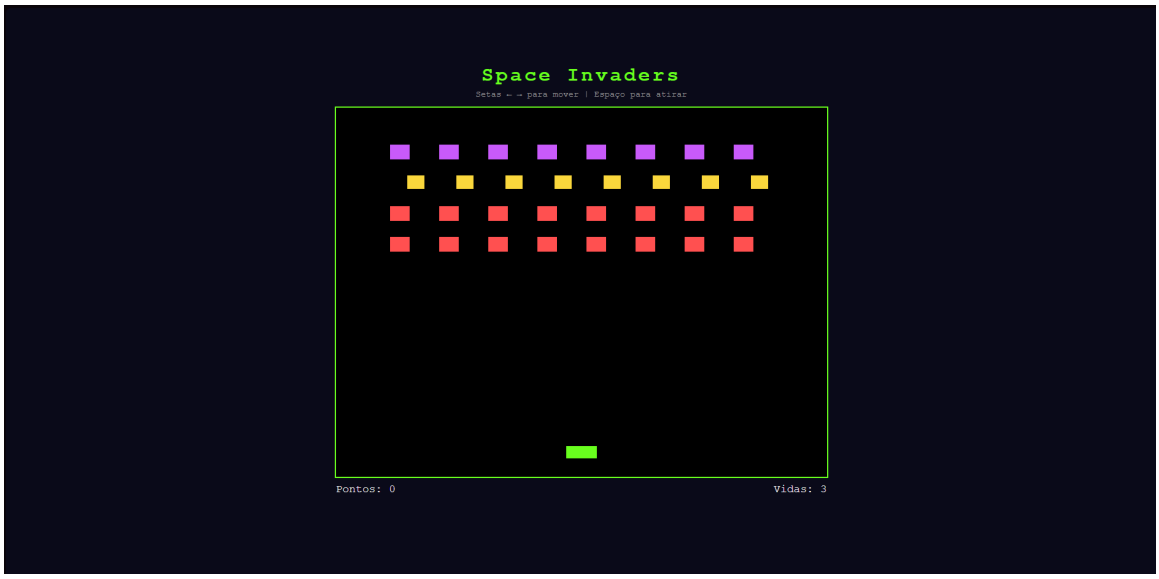
Como foi testado. Os testes rodaram no Chromium e passaram, sem erros no console:

Teste	Resultado
Ao abrir a página	Tela de início aparece e os inimigos ficam parados
Clicar em "Iniciar jogo"	A tela some, o jogo roda e o tiro funciona
Vidas chegam a zero	Aparece "GAME OVER", o motivo e a pontuação final
Invasores chegam até a nave	Aparece "GAME OVER" com o motivo "Os invasores chegaram até você!"
Todos os inimigos destruídos	Aparece "VOCÊ VENCEU!" com a pontuação final
"Jogar novamente" ou Enter	Pontos zerados, 3 vidas e 32 inimigos novos
Três reinícios seguidos	A velocidade dos inimigos continua a mesma, ou seja, há um único loop
Enter durante a partida	Não reinicia nem zera a pontuação

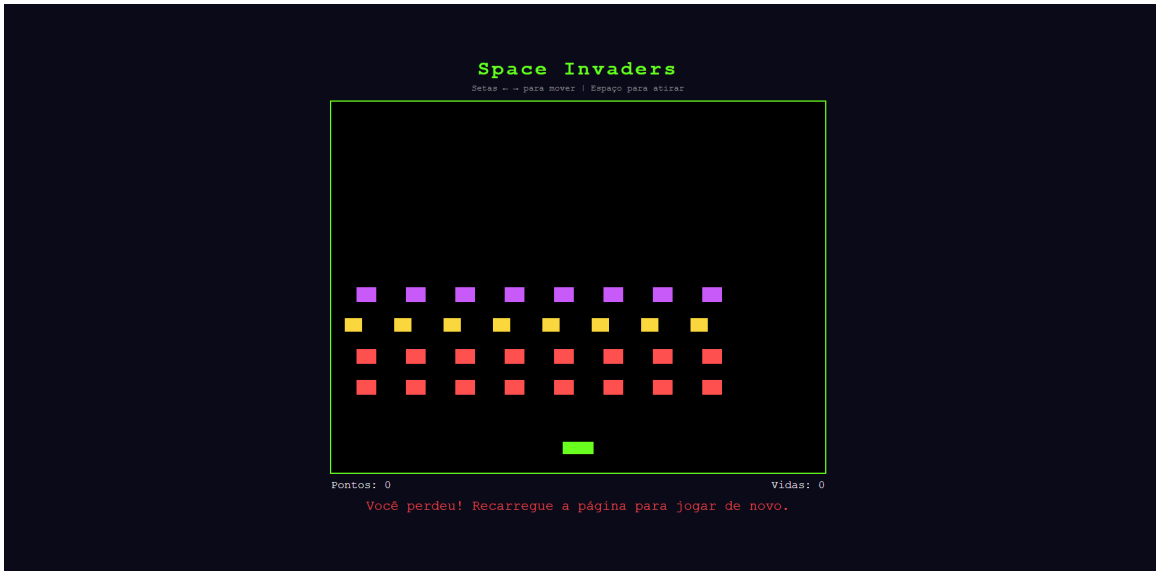
Pontos para explicar na apresentação:

- **Encapsulamento:** a classe `Tela` guarda os elementos da página e só oferece `mostrarInicio()`, `mostrarFim()` e `esconder()`. O `Jogo` não mexe nesses elementos diretamente.
- **Responsabilidade única:** o `Jogo` cuida das regras e a `Tela` cuida de mostrar mensagens.
- **Função como parâmetro:** o `Jogo` passa `() => this.iniciar()` para a `Tela`, que o chama quando o botão é clicado.
- **Por que `iniciar()` verifica `this.rodando`:** sem isso, clicar duas vezes ou apertar Enter com o botão em foco criaria dois loops e o jogo andaria mais rápido.
- **Por que `_resetarEstado()` recria os objetos:** recomeçar com os objetos da partida anterior manteria inimigos mortos, pontos e teclas presas.

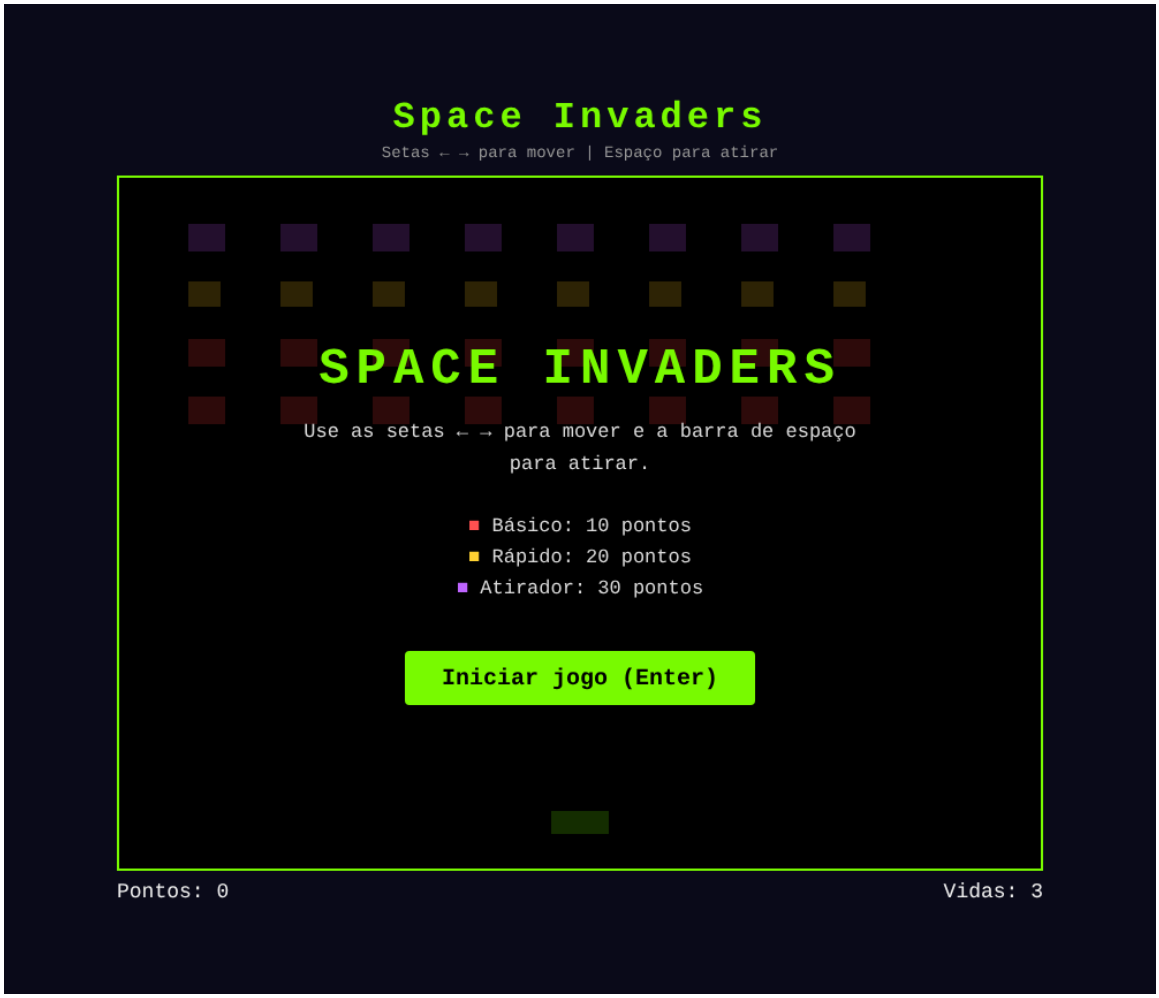
Evidências, antes da alteração. Na primeira versão o jogo já aparecia em andamento, sem tela de início:



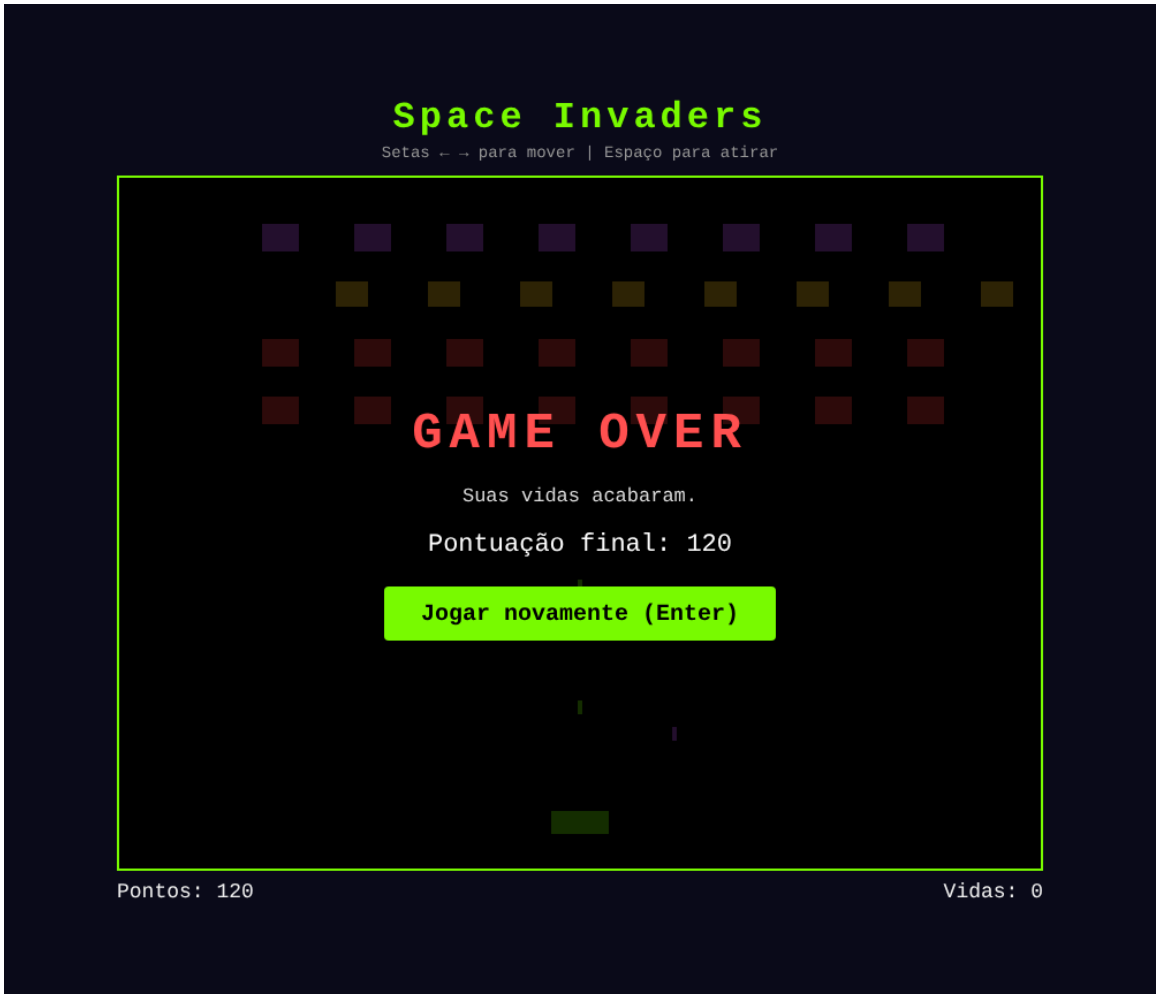
Ao perder, só aparecia um texto pedindo para recarregar a página, sem botão de reiniciar:



Evidências, depois da alteração. Tela de início, com instruções, legenda de pontos e botão "Iniciar jogo":



Tela de derrota, com motivo, pontuação final e botão "Jogar novamente":



Tela de vitória, com pontuação final e botão "Jogar novamente":



- ☐ Confirmar se usou outra IA além do Claude nesta parte

Conceitos de POO no código

O jogo usa classes, herança em dois níveis e polimorfismo por sobrescrita de métodos.

Conceito	Onde aparece
Classes e objetos	Cada arquivo `.js` define uma classe; o `Jogo` cria objetos com `new` (a nave, 32 inimigos, os projéteis)
Herança, nível 1	`Jogador`, `Inimigo` e `Projetil` herdam de `Entidade` e reaproveitam posição, tamanho, `desenhar()` e `colideCom()`
Herança, nível 2	`InimigoBasico`, `InimigoRapido` e `InimigoAtirador` herdam de `Inimigo`

Conceito	Onde aparece
Polimorfismo	O `Jogo` chama `mover()` e `tentarAtirar()` em qualquer inimigo, e cada subclasse responde do seu jeito
Sobrescrita	`InimigoRapido.mover()`, `InimigoAtirador.tentarAtirar()` e `Jogador.atualizar()` substituem o comportamento da classe pai
Encapsulamento	Cada classe guarda seus próprios dados (vida, velocidade, intervalo de tiro) e expõe métodos para usá-los

Classe base (herança):

```
class Entidade {
  constructor(x, y, largura, altura, cor) {
    this.x = x;
    this.y = y;
    this.largura = largura;
    this.altura = altura;
    this.cor = cor;
    this.ativo = true;
  }
}

class Inimigo extends Entidade { /* ... */ }
class InimigoRapido extends Inimigo { /* ... */ }
```

Sobrescrita em duas subclasses:

```
// InimigoRapido: mesmo método, comportamento diferente (mais veloz)
mover(velocidadeBase, larguraCanvas) {
  this.x += velocidadeBase * this.multiplicadorVelocidade * this.direcao;
}

// InimigoAtirador: só ele realmente dispara
tentarAtirar(listaProjeteis) {
  if (Math.random() < this.chanceDeAtirar) {
    const centroX = this.x + this.largura / 2 - 2;
    listaProjeteis.push(new Projetil(centroX, this.y + this.altura, 1, '#c266ff'));
  }
}
```

Chamada polimórfica no `Jogo.js` (a lista mistura os três tipos, e o código não precisa saber qual é qual):

```
this.inimigos.forEach((inimigo) => {
  inimigo.direcao = this.direcaoBloco;
  inimigo.mover(this.velocidadeBaseInimigos, this.canvas.width);
  inimigo.tentarAtirar(this.projeteisInimigo);
});
```

Créditos e referências

O código do Claude não usa imagens nem sons de terceiros: nave, inimigos e tiros são retângulos coloridos desenhados no `<canvas>`.

- Ferramentas de IA: Claude (Anthropic) e Gemini (Google)
- Editor: Visual Studio Code, com a extensão Live Server
- Enunciado: Trabalho da NI1, Programação Orientada a Objetos, turma 2NAADS
- ☐ Se o grupo usar assets, tutoriais ou trechos de terceiros, listar aqui com nome e link, como o enunciado exige